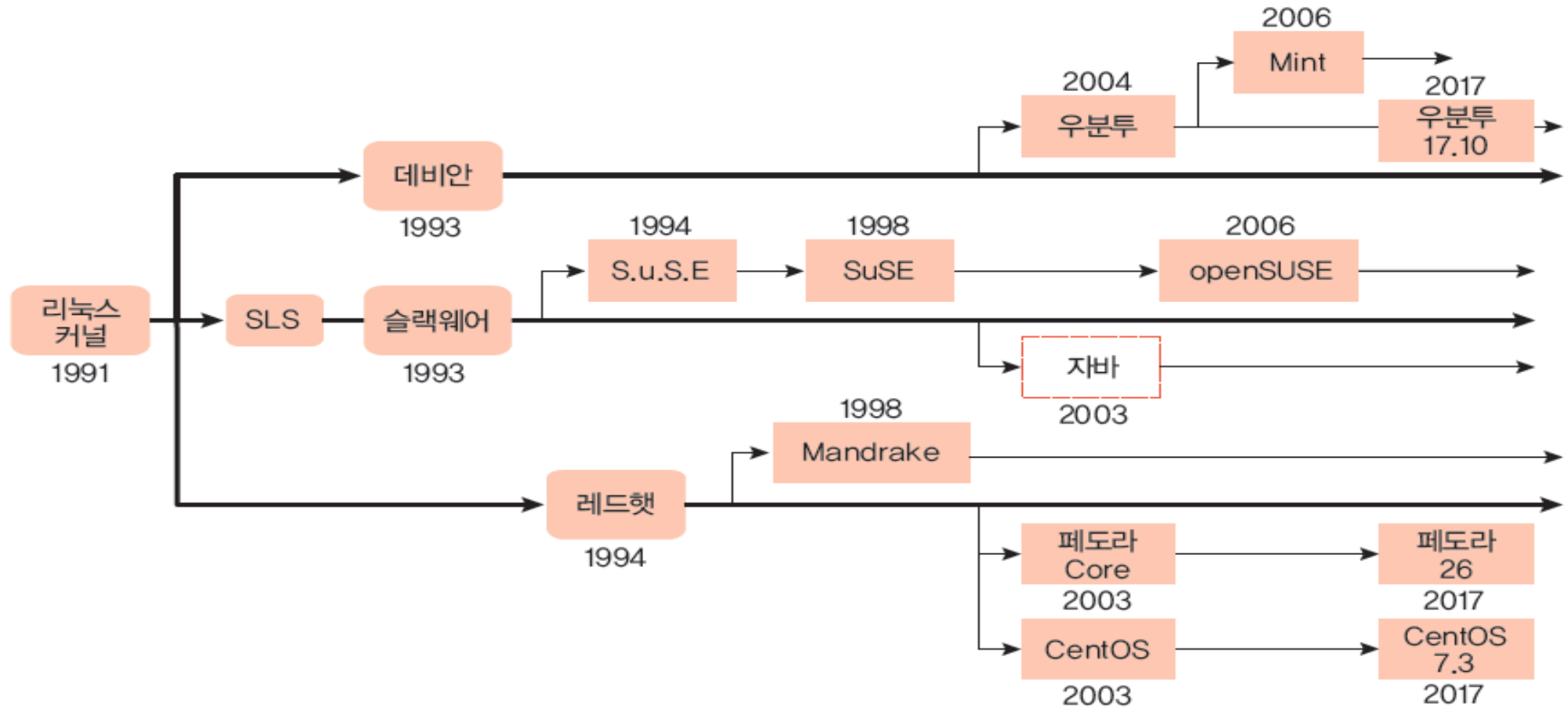


Linux

- Linux & 터미널

Linux 계통도

- Debian
- Slackware
- Red Hat



Linux의 특징

- Linux는 공개 소프트웨어이며 무료로 사용
- 유닉스와의 완벽한 호환성을 유지
- 서버용 운영체제로 많이 사용
- 편리한 GUI 환경을 제공

Linux의 구조

- Kernel: Linux의 핵심

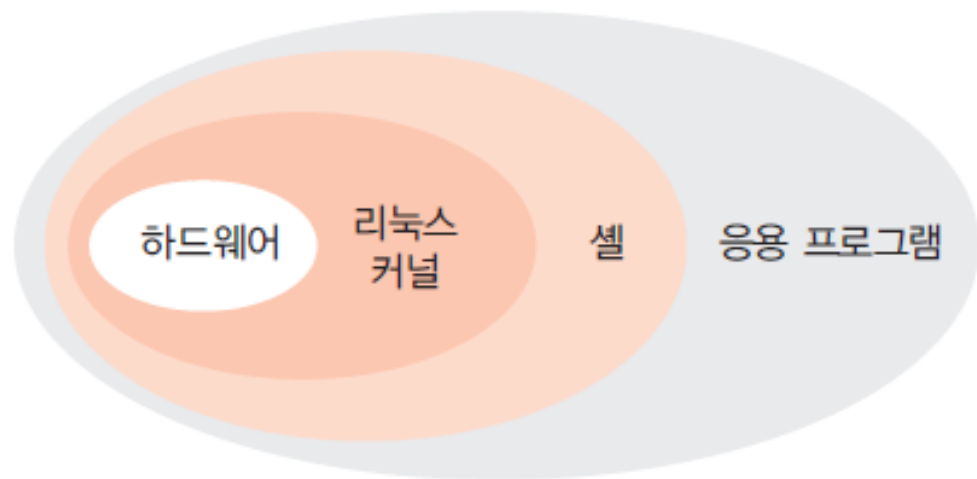
- 프로세스/메모리/파일 시스템/장치 관리
- 컴퓨터의 모든 자원 초기화 및 제어 기능

- Shell: 사용자 인터페이스

- 명령 해석
- 프로그래밍 기능
- Linux 기본 셸: 배시 셸 (Linux 셸)

- Application

- 각종 프로그래밍 개발 도구
- 문서 편집 도구
- 네트워크 관련 도구 등



Shell

- 셸은 사용자가 입력한 명령을 해석해 커널로 전달하거나 커널의 처리 결과를 사용자에게 전달하는 역할을 수행
- 명령을 입력하는 인터페이스
- 셸의 기능
 - 명령어 해석기
 - 사용자가 입력한 명령이나 파일에서 읽어 들인 명령을 해석하고 적절한 프로그램을 찾아서 실행
 - 프로그래밍
 - 셸은 자체 내에 프로그래밍 기능이 있어서 프로그래밍 가능
 - 셸의 프로그래밍 기능을 이용하면 여러 명령을 사용하여 반복적으로 수행하는 작업을 하나의 프로그램으로 만들 수 있는데 이렇게 작성된 셸 프로그램을 셸 스크립트라고 부름
 - 사용자 환경 설정
 - 셸은 사용자 환경을 설정할 수 있도록 초기화 파일 기능을 제공
 - 초기화 파일에는 명령을 찾아오는 경로를 설정하거나 파일과 디렉터리를 새로 생성할 때 적용하는 기본 권한을 설정하거나 다양한 환경 변수 등을 설정할 수 있음
 - 사용자가 로그인할 때 이 초기화 파일이 실행되어 사용자 별로 특성에 맞게 초기 환경이 설정됨

Shell

- 프롬프트

- 명령어를 입력할 때 보여지는 문자열

root@server01:~\$

- root는 사용자 이름

- server01은 호스트 이름

- 슈퍼 사용자인 경우는 \$ 대신에 #
- 리눅스에 사용자가 로그인을 하면 자동으로 셸이 시작하는데 로그인 후 처음으로 시작되는 셸이 로그인 셸
- 로그인 셸을 확인하는 명령은 echo \$SHELL
- 명령어를 입력하고 그 결과를 보면서 셸을 사용하는데 입력하고 그 결과를 바로 확인하는 조작 방식을 대화형(Interactive) 방식이라고 함
- 실행하고자 하는 명령어들을 미리 파일에 기록을 해두고 그 파일을 함께 셸에 넘겨주는 방식으로 명령을 수행하는 방식을 스크립트 방식이라고 함
- ~ 기호는 사용자의 홈 디렉터리
- 기본 디렉터리는 사용자의 홈 디렉터리에 위치

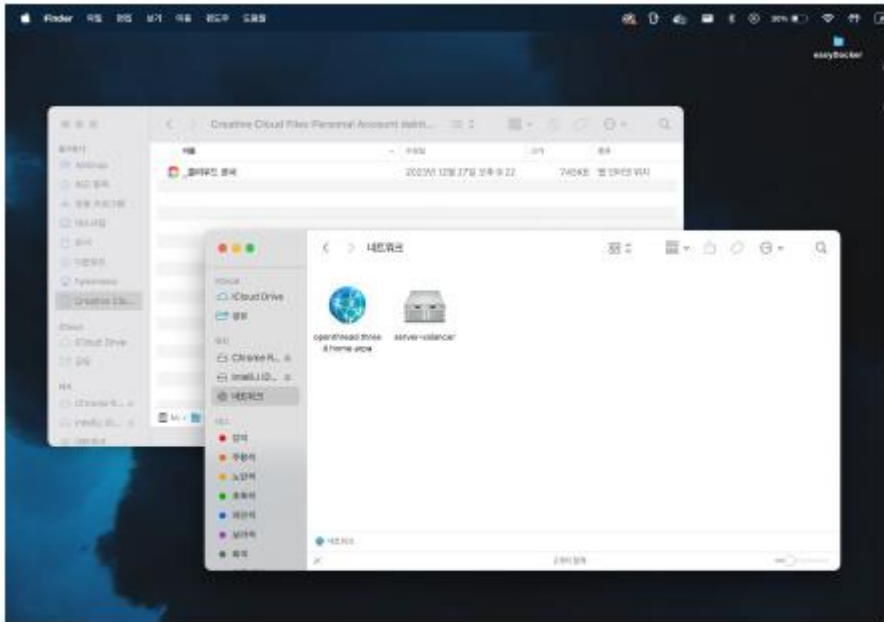
터미널은 무엇일까요?



```
~$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED
NAMES
b1e4de0d67b5   nginx                               "/docker-entrypoint..." 10 days
infallible_neumann
89fa57535744   postgres:13                        "docker-entrypoint.s..." 2 weeks
my-containerB
e4f2159f7232   postgres:13                        "docker-entrypoint.s..." 2 weeks
my-containerA
7508dbb2f4b5   5b79ff60fb24                       "java -jar leafy.jar"     3 weeks
localLeafyDevContainer
53be6a040cda   devwikirepo/leafy-postgres:1.0.0  "docker-entrypoint.s..." 3 weeks
postgres

~$ docker image ls
REPOSITORY          TAG          IMAGE ID          CREATED          SIZE
dev                 latest       5b79ff60fb24     3 weeks ago     25
<none>              <none>       25f330be7ecf     3 weeks ago     76
<none>              <none>       fa09649fded1     3 weeks ago     77
<none>              <none>       20d763cebc9a     3 weeks ago     76
<none>              <none>       14fdb62cc26      3 weeks ago     77
<none>              <none>       56b05a8c4db0     3 weeks ago     76
nginx              latest       760b7cbba31e     3 weeks ago     19
devwikirepo/leafy-postgres 1.0.0       ad6e1fdcd2a4     6 weeks ago     43
devwikirepo/memoryuse   latest       0249b48c87e2     2 months ago    10
```

터미널은 무엇일까요?



GUI

Graphical User Interface

```
~/ docker ps
CONTAINER ID   IMAGE                                COMMAND
NAMES
b1e4de0d67b5   nginx                               "/docker-e
infallible_neumann
89fa57535744   postgres:13                         "docker-er
my-containerB
e4f2159f7232   postgres:13                         "docker-er
my-containerA
7508dbb2f4b5   5b79ff60fb24                        "java -jar
localLeafyDevContainer
53be6a040cda   devwikirepo/leafy-postgres:1.0.0   "docker-er
postgres

~/ docker image ls
REPOSITORY    TAG       IMAGE ID
dev           latest    5b79ff60fb2
<none>        <none>    25f330be7ec
<none>        <none>    fa09649fdec
<none>        <none>    20d763cebc9
```

CLI

Command Line Interface

왜 터미널을 사용할까요?



git



CLI
Command Line Interface

왜 터미널을 사용할까요?



명령어로 작업



언제든지 재사용 가능



자동화에 용이

Linux

- 파일 시스템

파일 시스템

현재 위치를 확인하는 `pwd`

디렉토리를 만드는 `mkdir`

디렉토리의 파일 리스트를 조회하는 `ls`

디렉토리를 이동하는 `cd`

디렉토리의 구조를 확인하는 `tree`

디렉토리/파일을 복사하는 `cp`

디렉토리/파일을 이동하는 `mv`

디렉토리/파일을 삭제하는 `rm`

현재 위치를 확인하는 pwd

"**p**rint **w**orking **d**irectory"

```
$ pwd  
/Users/devwiki  
$ clear
```

디렉터리 만들기

디렉터리를 만드는

`mkdir [생성할디렉터리명]`

"make directory"

```
$ cd ~
```

```
$ mkdir easylinux
```

디렉터리 이동

디렉터리를 이동하는 `cd`

"change directory"

디렉터리 이동: `cd [이동할 디렉터리명]`

홈 디렉터리 이동: `cd ~`

상위 디렉터리 이동: `cd ..`

디렉터리 이동

디렉터리를 이동하는 **cd**

```
$ cd easylinux
```

```
$ pwd
```

```
/Users/devwiki/easylinux
```

```
$ mkdir alpha beta gamma delta
```

```
$ cd alpha
```

```
$ pwd
```

```
/Users/devwiki/easylinux/alpha
```

```
$ cd ..
```

```
$ pwd
```

```
/Users/devwiki/easylinux
```

```
$ cd beta
```

```
$ pwd
```

```
/Users/devwiki/easylinux/beta
```

```
$ cd ../gamma
```

```
$ pwd
```

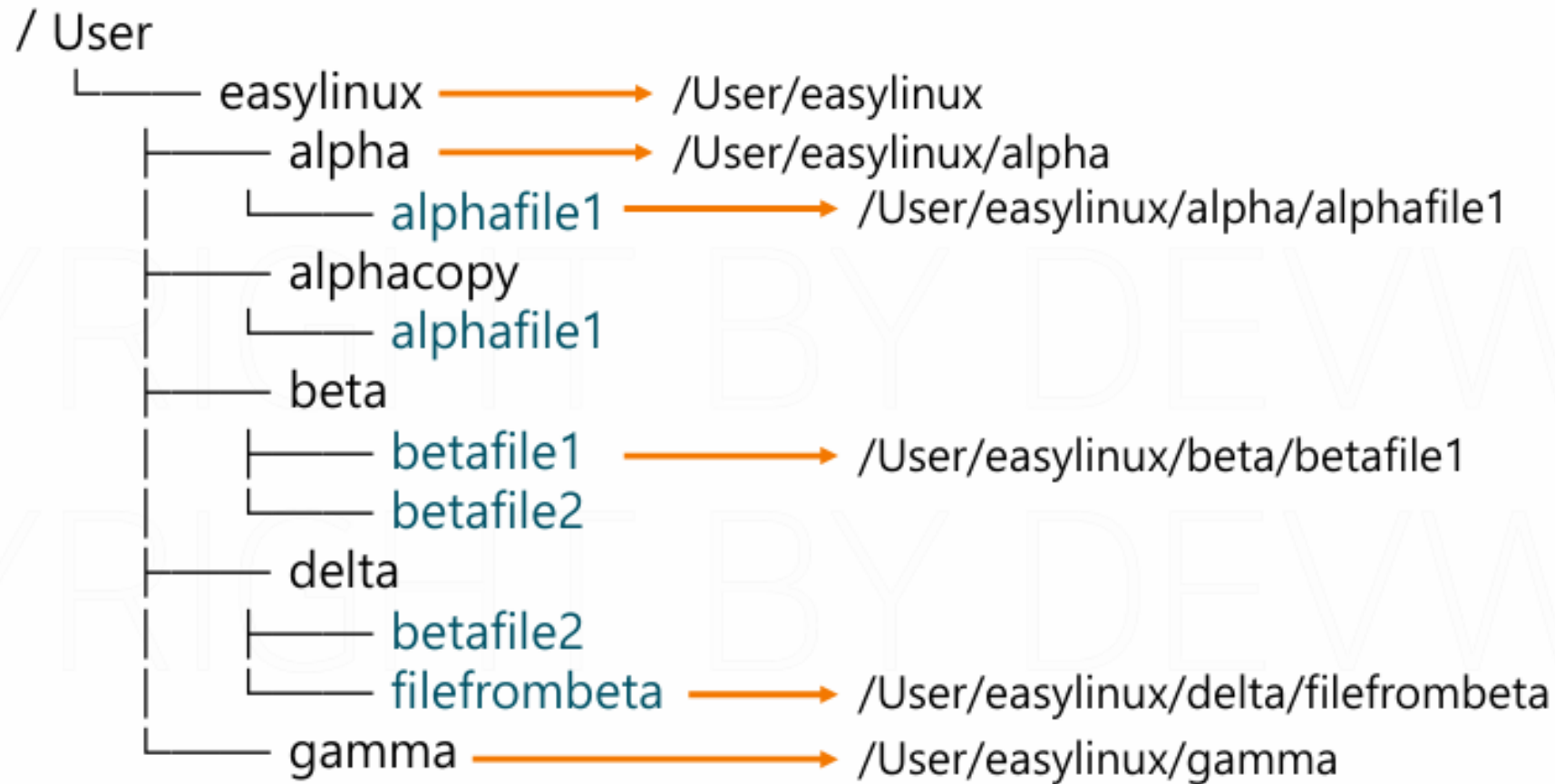
```
/Users/devwiki/easylinux/gamma
```

```
$ cd /Users/hyeonwoo/easylinux/delta
```

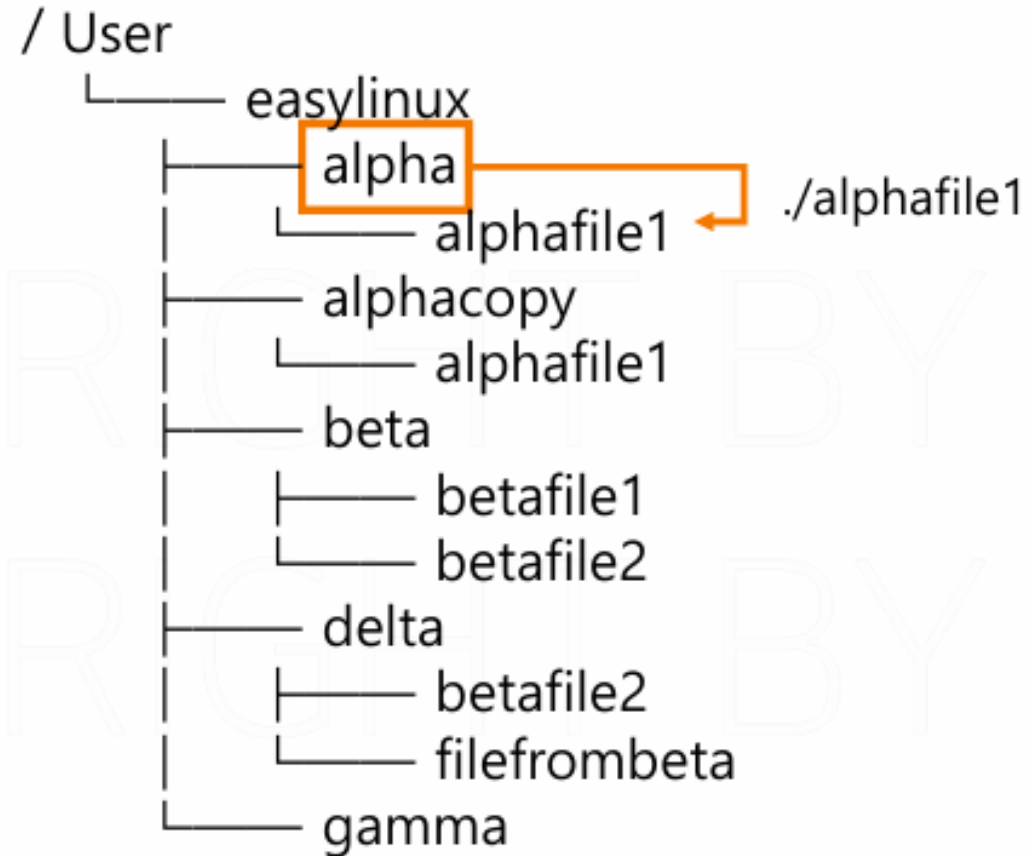
```
$ pwd
```

```
/Users/devwiki/easylinux/delta
```

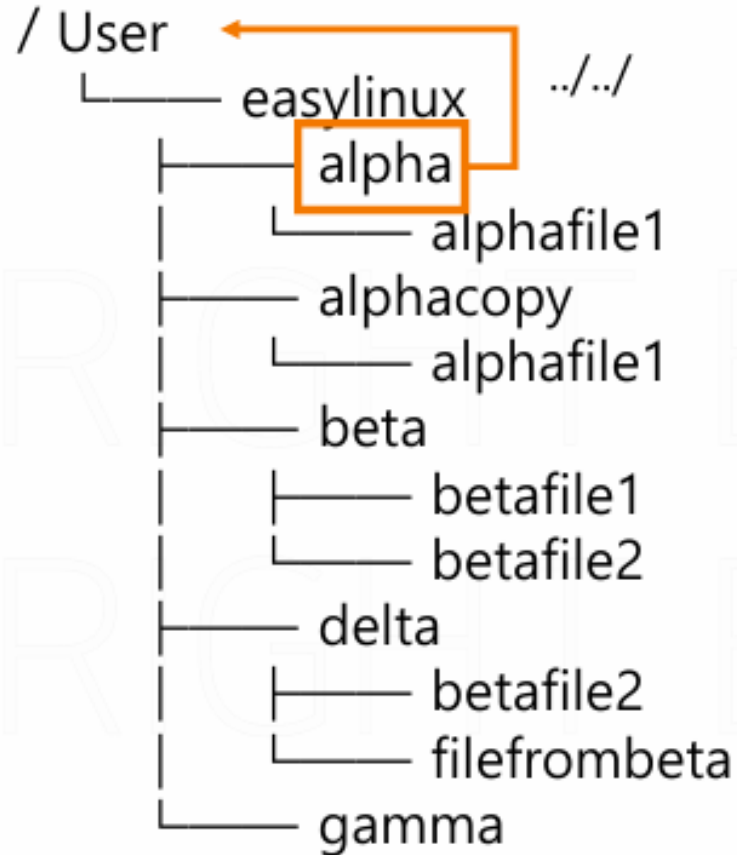
절대경로와 상대경로



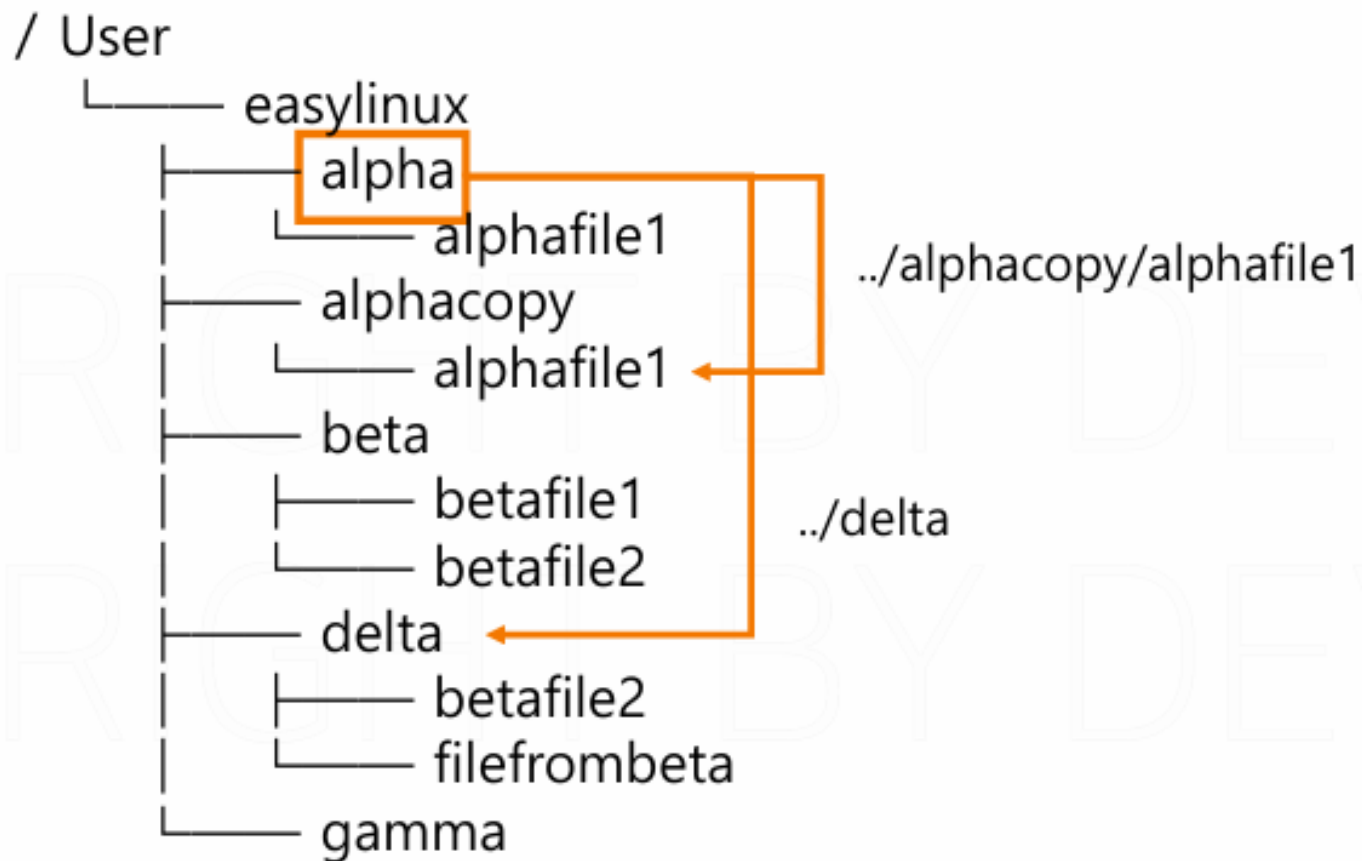
절대경로와 상대경로



절대경로와 상대경로



절대경로와 상대경로



디렉터리의 파일 리스트 조회

디렉터리의 파일 리스트를 조회하는 **ls**

"list"

현재 위치 기준 디렉터리 조회: **ls**

특정 디렉터리 조회: **ls [디렉터리 위치]**

숨김 파일 및 파일 사이즈도 함께 조회: **ls -alh**

디렉터리의 파일 리스트 조회

디렉터리의 파일 리스트를 조회하는 **ls**

```
$ ls ~/easylinux
alpha/ beta/ delta/ gamma/
$ cd ~/easylinux/alpha
$ touch alphafile1
$ touch .alphafile2
$ ls
alphafile1
$ ls -alh
drwxr-xr-x  4 staff  128B  3 19 21:25 .
drwxr-xr-x  6 staff  192B  3 19 21:24 ..
-rw-r--r--  1 staff    0B  3 19 21:25 .alphafile2
-rw-r--r--  1 staff    0B  3 19 21:25 alphafile1
```

```
$ ls -alh ~/easylinux
drwxr-xr-x  6 staff  192B  3 19 21:24 .
drwxr-x---+ 64 staff  2.0K  3 19 21:26 ..
drwxr-xr-x  4 staff  128B  3 19 21:25 alpha
drwxr-xr-x  2 staff   64B  3 19 21:24 beta
drwxr-xr-x  2 staff   64B  3 19 21:24 delta
drwxr-xr-x  2 staff   64B  3 19 21:24 gamma
```

디렉터리/파일의 구조 확인

디렉터리/파일의 구조를 확인하는 **tree**

일반 조회: **tree**

디렉터리만 조회: **tree -d**

숨김파일까지 조회: **tree -a**

깊이 지정 조회: **tree -L {깊이}**

디렉터리/파일의 구조 확인

디렉터리의 구조를 확인하는 **tree**

```
$ cd ~/easylinux
```

```
$ tree
```

```
|— alpha
|   └─ alphafile1
|— beta
|— delta
└─ gamma
```

```
$ tree -d
```

```
|— alpha
|— beta
|— delta
└─ gamma
```

```
$ tree -a
```

```
|— alpha
|   └─ .alphafile2
|   └─ alphafile1
|— beta
|— delta
└─ gamma
```

```
$ tree -L 1 /etc
```

/etc 아래의 1단계 까지만 조회

디렉터리/파일 복사

디렉터리/파일을 복사하는 **cp**

"copy"

파일 복사: **cp** [원본 파일] [대상 위치]

다른이름으로 복사: **cp** [원본 파일] [위치와 파일명]

디렉터리 복사: **cp -r** [원본 위치] [대상 위치]

현재 디렉터리로 복사: **cp -r** [원본 위치] .

디렉터리/파일 복사

디렉터리/파일을 복사하는 **cp**

```
$ cd ~/easylinux/beta
$ touch betafile1 && touch betafile2
$ tree -a ~/easylinux
├── alpha
│   ├── .alphafile2
│   └── alphafile1
├── beta
│   ├── betafile1
│   └── betafile2
├── delta
└── gamma

$ cp betafile2 ../delta
$ ls ../delta
betafile2
```

```
$ cp betafile2 ../delta/filefrombeta
$ ls ../delta
betafile2  filefrombeta

$ cp -r ~/easylinux/alpha ~/easylinux/alphacopy
$ tree -a ~/easylinux
├── alpha
│   ├── .alphafile2
│   └── alphafile1
├── alphacopy
│   ├── .alphafile2
│   └── alphafile1
├── beta
│   ├── betafile1
│   └── betafile2
├── delta
└── gamma

$ cp ../alpha/alphafile1 .
$ cp ../alpha/alphafile1 ./filefromalpha
```

디렉터리/파일 이동

디렉터리/파일을 이동하는 **mv**

"move"

파일/디렉터리 이동: **mv** [원본 위치] [새 위치]

디렉터리/파일 이동

디렉터리/파일을 이동하는 **mv**

```
$ cd ~/easylinux
$ ls
alpha      alphacopy beta      delta      gamma
$ mv delta/filefrombeta gamma
$ ls gamma
filefrombeta
$ mv alpha/alphafile1 gamma/filefromalpha
$ ls gamma
filefromalpha filefrombeta
$ ls -alh alpha
-rw-r--r--  1 staff   0B   3 19 21:25 .alphafile2
```

```
$ mv beta/* gamma
alphafile1 betafile1 betafile2 filefromalpha
filefrombeta
$ mv gamma/betafile1 gamma/gammafile1
$ mv gamma/betafile2 gamma/gammafile2
$ ls gamma
alphafile1 filefromalpha filefrombeta gammafile1
gammafile2
$ mv alphacopy alphamove
$ ls
alpha      alphamove beta      delta      gamma
```

디렉터리/파일 찾기

디렉터리/파일을 찾는 **find**

파일/디렉터리 찾기:

find [찾을 위치] -name [파일/디렉터리명]

디렉터리/파일 찾기

디렉터리/파일을 찾는 find

```
$ cd ~/easylinux
$ tree
~/easylinux/ tree
.
├─ alpha
├─ alphamove
│   └─ alphafile1
├─ beta
├─ delta
│   └─ betafile2
└─ gamma
    ├── alphafile1
    ├── filefromalpha
    ├── filefrombeta
    ├── gammafile1
    └─ gammafile2
```

```
$ find ~/easylinux -name gammafile1
/Users/easylinux/gamma/gammafile1
```

```
$ find ~/easylinux/beta -name gammafile1
```

디렉터리/파일 삭제

디렉터리/파일을 삭제하는 **rm**

"remove"

파일 삭제: **rm** [삭제할 파일 위치]

디렉터리 삭제: **rm -r** [삭제할 디렉터리 위치]

파일 강제 삭제(*주의): **rm -rf** [삭제할 파일 위치]

디렉터리/파일 삭제

디렉터리/파일을 삭제하는 **rm**

```
$ cd ~/easylinux
$ ls
alpha      alphamove beta      delta      gamma
$ rm alpha
rm: alpha: is a directory
$ rm -r alpha
$ ls
alphacopy beta      delta      gamma
$ cd gamma
$ ls
alphafile1 filefromalpha filefrombeta gammafile1
gammafile2
```

```
$ rm -rf gammafile1 gammafile2
$ ls
alphafile1 filefromalpha filefrombeta
$ rm -rf filefrom*
$ ls
alphafile1
$ cd ..
$ ls
alphamove beta      delta      gamma
$ rm -rf *
$ ls
```

Linux

- 파일 작성 / 수정

파일을 작성하고 수정해보기

vi 편집기 사용하기

파일의 내용을 확인하는 **cat**

출력 내용을 검색하는 **grep**

출력 결과를 저장하는 리다이렉션 **>, >>**

큰 파일을 읽을때는 **less**

아카이브로 묶는 **tar**

vi 편집기 사용하기

INSERT 모드(i): 문자를 편집할 수 있습니다.

COMMAND 모드(esc): 복사/붙여넣기,
파일 저장/종료 등 다양한 작업을 할 수 있습니다.

vi 편집기 사용하기

COMMAND 모드에서 할 수 있는 것들(기본)

종료(**q**uit) : **q** 입력 후 enter

저장(**w**rite) 및 종료 : **wq** 입력 후 enter

저장 및 강제종료 (!) : **wq!** 입력 후 enter

*커맨드모드에서는 영문만 사용할 수 있습니다.

vi 편집기 사용하기

```
$ cd ~/easylinux
```

```
$ vi myfirstfile
```

1. i 키를 사용해 INSERT 모드로 전환
(하단에 --INSERT-- 표시가 나타납니다.)
2. 'Hello Vi!' <- 입력
3. ESC를 눌러서 명령(Command) 모드로 전환
4. :wq 입력 후 엔터를 눌러 파일 저장 및 종료

```
$ vi myfirstfile
```

1. i 키를 사용해 INSERT 모드로 전환
2. Hello Vi! 다음에 add text <- 입력
3. ESC를 눌러서 명령(Command) 모드로 전환
4. :q 입력 후 엔터를 눌러 vi 종료 시도
5. :q! 입력 후 엔터를 눌러 vi 종료

```
$ vi myfirstfile
```

1. 파일이 변경되지 않는 것을 확인
2. :q 입력 후 엔터를 눌러 vi 종료

vi 편집기 사용하기

COMMAND 모드에서 할 수 있는 것들(활용)

줄삭제(delete) dd

복사(copy) yy

붙여넣기(paste) p

되돌리기(undo) u

*커맨드모드에서는 영문만 사용할 수 있습니다.

vi 편집기 사용하기

```
$ cd ~/easylinux
```

```
$ vi Dockerfile
```

1. i 키를 사용해 INSERT모드로 전환
2. 강의자료의 Dockerfile 복사 후 붙여넣기 후 저장
3. ESC를 눌러서 명령(Command) 모드로 전환
4. :wq 입력 후 엔터를 눌러 파일 저장 및 종료

```
$ vi Dockerfile
```

1. 4번째 COPY 라인에서 dd입력하여 1줄 제거
2. u를 입력하여 제거 취소
3. 3번째 LABEL 라인에서 yy 입력으로 복사
4. 맨 끝줄로 이동하여 p 입력으로 붙여넣기
5. :q! 입력 후 엔터를 눌러 vi 종료

vi 편집기 사용하기

COMMAND 모드에서 할 수 있는 것들(고급)

라인 보기 :set number

텍스트 찾기 /검색어 후 n 으로 탐색

텍스트 대체 :%s/원본/대체/g

*커맨드모드에서는 영문만 사용할 수 있습니다.

vi 편집기 사용하기

```
$ vi nginx.conf
```

1. i 키를 사용해 INSERT모드로 전환
2. 강의자료의 nginx.conf 텍스트 복사 후 붙여넣기
3. ESC를 눌러서 명령(Command) 모드로 전환
4. :set number 로 라인 출력
5. /html 검색후 n을 사용해서 이동
6. /expires 검색후 INSERT모드에서 30d -> 12h로 변경
7. ESC를 눌러서 명령(Command) 모드로 전환
8. :%s/example.com/devwiki.com/g
9. :wq! 로 저장 및 종료

파일 내용 확인

파일의 내용을 확인하는 **cat**

```
$ cat myfirstfile  
...(myfirstfile의 내용 출력)
```

```
$ cat Dockerfile  
...(Dockerfile의 내용 출력)
```

```
$ cat nginx.conf  
...(nginx의 내용 출력)
```

출력 내용 검색

출력 내용을 검색하는 **grep**

파이프 '|' 과 함께 사용합니다

[이전 명령어] | [이후 명령어]
명령어의 출력을 다음 명령으로 전달

출력 내용 검색

출력 내용을 검색하는 **grep**

[출력명령어] | grep [검색어]

파일의 내용 검색: cat [파일명] | grep [검색어]

이전 n라인 출력: grep -B n [검색어]

이후 n라인 출력: grep -A n [검색어]

대소문자 구분 없이 검색: grep -i [검색어]

검색어에 공백이 있을 경우 '검색어' 로 묶어서 입력합니다.

출력 내용 검색

출력 내용을 검색하는 **grep**

```
$ cat Dockerfile | grep CMD
CMD ["nginx", "-g", "daemon off;"]
```

```
$ cat nginx.conf | grep location
    location / {
    location = /50x.html {
    location ~*
    \.(jpg|jpeg|png|gif|ico|css|js)$ {
```

```
$ cat Dockerfile | grep -i -B 2 'cmd'
EXPOSE 80
COPY . /usr/share/nginx/html
CMD ["nginx", "-g", "daemon off;"]
```

```
$ cat nginx.conf | grep -A 2 -B 3 'access_log
off;'
    location ~* \.(jpg|jpeg|png|gif|ico|css|js)$ {
        expires 12h;
        access_log off;
    }
}
```

출력 결과 저장

출력 결과를 저장하는 리다이렉션 >, >>

[이전 명령어] > 파일명

이전 명령어의 출력을 파일로 저장

> : 기존 파일이 있을경우 덮어쓰기

>> : 기존 파일이 있을경우 내용추가

출력 결과 저장

출력 결과를 저장하는 리다이렉션 >, >>

```
$ cd ~/easylinux
$ tree /etc | grep sh
(tree /etc 명령어 결과 중 sh 검색)
$ tree /etc | grep sh > tree_result.txt
$ cat tree_result.txt
(tree /etc 명령어 결과 내용 출력)
$ cat nginx.conf
(nginx.conf 파일 내용 출력)
$ cat nginx.conf > tree_result.txt
$ cat tree_result.txt
(nginx.conf 내용이 덮어씌워진 파일 내용 출력)
```

```
$ tree /etc >> tree_result.txt
$ cat tree_result.txt
(nginx.conf 내용에 tree 명령어 결과가 추가된
내용 출력)
```

less

큰 파일을 읽을때는 less

```
$ less tree_result.txt
```

Enter 및 Space로 탐색

q 로 종료

명령어	내용
<code>[Space]</code> , <code>[f]</code> , <code>[Ctrl] + [v]</code>	한 화면 아래로 스크롤
<code>[b]</code> , <code>[Meta] + [v]</code>	한 화면 위로 스크롤
<code>[j]</code> , <code>[Ctrl] + [n]</code> , <code>[Enter]</code>	한 행 아래로 스크롤
<code>[k]</code> , <code>[Ctrl] + [p]</code>	한 행 위로 스크롤
<code>[q]</code>	less 명령어 종료

- less 명령어로 스크롤 조작하기

tar

아카이브로 묶는 **tar**

아카이브란?

여러 파일이나 디렉토리를
하나의 파일로 묶음



아카이브
(tar)

데이터를 더 작은 크기로
줄이는 과정 포함



압축
(zip, gz..)

tar

아카이브로 묶는 **tar**

파일이나 디렉터리 아카이브:

: **tar -cvf** [결과파일.tar] [디렉터리또는파일]

파일이나 디렉터리 아카이브 해제:

: **tar -xvf** [결과파일.tar]

tar

파일을 아카이빙하는 **tar**

```
$ cd ~/easylinux
$ ls
Dockerfile      myfirstfile
tree_result.txt nginx.conf
$ tar -cvf easylinux.tar .
a ./Dockerfile
a ./tree_result.txt
a ./nginx.conf
a ./myfirstfile
$ mkdir easylinux_tar
$ cd easylinux_tar
$ mv ../easylinux.tar .
```

```
$ ls
easylinux.tar
$ tar -xvf easylinux.tar
x ./
x ./Dockerfile
x ./tree_result.txt
x ./nginx.conf
x ./myfirstfile
$ ls
Dockerfile      myfirstfile      tree_result.txt
easylinux.tar   nginx.conf
```

tar

아카이브를 압축하는 **tar**

파일이나 디렉터리 압축:

: **tar -czvf** [압축파일.tar.gz] [디렉터리또는파일]

파일이나 디렉터리 압축 해제:

: **tar -xzf** [압축파일.tar.gz]